



Concert Table Reservation System for Restaurant

Group Members

SE 101

Chetsada Winaikoson	6772012221
Waris Natirutthakorn	6872081621
Natcha Thawinpipat	6870083521
Watayut Aiamanan	6970267821

This document is an integral component of the subject

2110521 Software Architecture

Department of Software Engineering, Faculty of Engineering, Chulalongkorn University

Semester 1 of the academic year 2026



Problem Description

Pubs and bars still rely on manual chat messaging and phone calls to handle table reservations, leading to delayed staff responses, lost bookings, lack of real-time floor plan visibility, and frequent allocation errors where customers do not get their requested tables. Currently, handling table inquiries and confirming seat statuses manually consumes substantial staff time and often leads to miscommunications during peak hours. The system should streamline table reservations through a web app accessed via LINE Messenger, providing a real-time interactive floor plan, digital QR tickets for check-in, and automated cutoff reminders with arrival confirmation and extension requests.

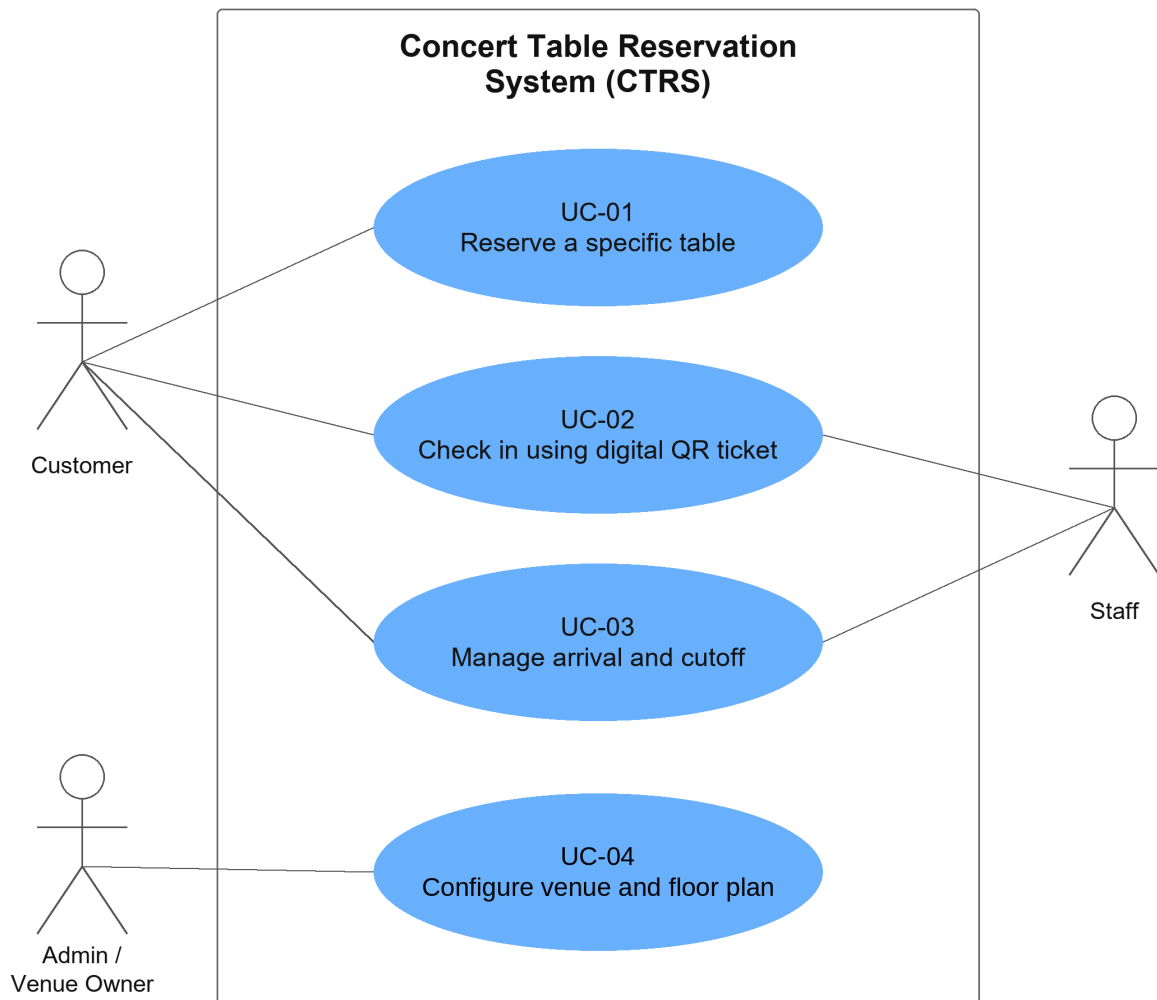
Target Customer

- **Manager (Bar/Venue Owner):** Full access to create the venue zone map (zones, tables, and table types), create and publish concert rounds with their prices, rule on check-in escalations, monitor booking traffic, and view booking analytics.
- **Staff (Front-of-House / Host):** Scan QR codes on digital tickets for fast check-in, monitor real-time table statuses, handle walk-ins, and create cutoff/postponement requests.
- **Customer:** Browse interactive floor plans, reserve specific tables with seat view perspectives, receive automated cutoff reminders, and check in via digital QR pass.



Scenario (use-case & description)

Use Case Diagram





Use case description

Use Case Name: Reserve a Specific Table	ID: UC-01	Importance Level: High
Primary Actor: Customer	Use Case Type: Business / Transactional	
Stakeholders and Interests: Customer: Wants to select a suitable table and receive reliable booking confirmation. Staff: Wants accurate reservations with fewer manual inquiries and table allocation errors. Manager: Wants efficient table utilization and visibility into bookings. Payment Gateway: Wants correct payment requests and reliable delivery of payment results.		
Brief Description: Customer accesses the web app through LINE Messenger, browses the upcoming concert rounds, views the real-time zone map and holds a specific table for 15 minutes. After completing the profile, accepting the booking terms and paying the full table fee through the Payment Gateway, the system confirms the booking, issues an e-ticket with a QR code and sends it to the Customer by LINE.		
Trigger: Customer wants to reserve a table for a concert round. Type: External		
Relationships: Relies on LINE Login for customer authentication, the LINE Messaging API for confirmation messages, the Payment Gateway for payment, and the concert round and zone map created in UC-03. Produces the Confirmed booking and e-ticket used in UC-02; the check-in window and grace period applied in UC-02 come from the round created in UC-03.		
Precondition: - Customer has the LINE application and is a friend of the shop's LINE Official Account. - Manager has created the concert round with its zone map, table types, package prices and booking-open time (UC-03). - The payment gateway merchant account is active and the shop's bank account for degraded mode is configured.		
Postcondition: Success (basic flow; EF-3 slip approved; EF-6): - Booking state = Confirmed; the payment is recorded; the table is booked for the round; the e-ticket is issued; the confirmation has been sent by LINE or is retrievable under My Bookings. Cancelled (AF-4 cancel; AF-6 consent refused; EF-5 login failed):		



- Booking state = Cancelled (or no booking was created); the hold is released and the table is available again; no payment exists.

Expired (EF-1 hold expired; EF-3 slip rejected or not attached; EF-4 hold ended before a result):

- Booking state = Expired; the table is available again and the zone map is refreshed; no money is held from the **Customer**; the **Customer** has been informed on screen and by LINE.

Refunded (EF-2 payment result after hold expired, table no longer available):

- Booking state = Expired; the payment and the refund are both recorded; the table remains with the other customer; the **Customer** has been informed by LINE that the payment is being returned.

Normal Flow of Events:

{Open the Customer Frontend}

1. The use case begins when the **Customer** opens the Rich Menu of the LINE Official Account and chooses "Reserve a table".
2. **System** signs the **Customer** in through LINE Login and uses the LINE user id as the customer identity. {LINE Login Result}

{Browse Rounds}

3. **System** displays the upcoming concert rounds with artist, date, start time, booking-open time and status: not yet open, open, sold out.
4. **Customer** selects a round that is open for booking.

{View the Table Map}

5. **System** displays the zone map of the round: every table with its number, zone, table type, package price and status (available, held, booked), refreshed in real time.
6. **Customer** selects an available table.

{Hold the Table}

7. **System** verifies that the table is still available and places a hold on it for the hold period of 15 minutes.
8. **System** marks the table as held for every other customer (first lock wins) and displays the remaining hold time.
9. **System** shows the booking summary (round, table, package content and package price) and asks for the party size.
10. **Customer** enters the party size.
11. **System** computes and displays the full table fee: the package price of the selected table type in its zone plus the extra-person fee for every person above the capacity of the table type.



{Complete the Customer Profile}

12. **System** checks whether a customer profile already exists for the LINE user id.
 - a. If this is the **Customer's** first booking, **System** shows the purpose of data collection and asks for consent, name and phone; **Customer** provides them and **System** stores the customer profile.
 - b. Otherwise **System** pre-fills the profile and **Customer** confirms it.

{Accept the Booking Terms}

13. **System** displays the booking terms: full payment confirms the booking, the check-in window opens 2 hours before the show, the grace period is 30 minutes after the start, and a no-show is not refunded.
14. **Customer** accepts the terms.

{Pay the Full Table Fee}

15. **System** creates a payment request for the full table fee with the **Payment Gateway** and opens the gateway's hosted checkout inside the web app.
16. **Customer** chooses a payment method (PromptPay QR, card, mobile banking, e-wallet).
17. **Customer** submits the payment through the chosen method before the hold expires.

{Payment Completed}

18. **Payment Gateway** verifies the payment and sends the payment result to **System** by signed webhook. **{Payment Result Received}**

{Confirm the Booking}

19. **System** verifies the authenticity of the payment result and the amount, records the payment, sets the booking to Confirmed and marks the table as booked; if the same payment result arrives again, **System** ignores the duplicate. **{Booking Confirmed}**

{Issue the E-Ticket}

20. **System** issues the e-ticket: it generates a signed booking reference valid only for that concert round and table, encodes it in a QR code, stores it with the booking, and makes it available under My Bookings.
21. **System** displays the confirmation and the e-ticket on screen.
22. **System** sends the confirmation message with the e-ticket and the booking terms to the **Customer** through the LINE Messaging API (subflow **{Notify the Customer by LINE}**).

{Confirmation Delivery Result}

23. The use case ends.

Sub flows:

{Notify the Customer by LINE}

1. **System** composes the message from the booking data and the booking terms template.



2. **System** sends it as a push message through the LINE Messaging API and records the delivery result.
3. The subflow returns to the step that invoked it.

Alternate/Exceptional Flow:

Alternative Flows:

AF-1 Round Not Yet Open for Booking

At **{Browse Rounds}**, if the **Customer** selects a round whose booking-open time has not arrived,

1. **System** shows the round's details and its booking-open time and does not display the zone map.
2. Resume the basic flow at **{Browse Rounds}**.

AF-2 Round Sold Out

At **{Browse Rounds}**, if the **Customer** selects a round whose status is sold out, or at **{View the Table Map}**, if the last available table of the round is taken while the map is displayed,

1. **System** shows the round as sold out and closes the zone map if it is displayed.
2. Resume the basic flow at **{Browse Rounds}**.

AF-3 Table Just Taken by Another Customer

At **{Hold the Table}**, if the verification finds that the selected table has already been held or booked by another customer (first lock wins),

1. **System** informs the **Customer** that the table has just been taken and refreshes the zone map.
2. Resume the basic flow at **{View the Table Map}**.

AF-4 Customer Cancels During the Hold

At any point between **{Hold the Table}** and **{Payment Completed}**, if the **Customer** cancels the booking,

1. **System** releases the hold immediately, sets the booking to Cancelled and returns the table to available.
2. The use case ends.

AF-5 Payment Declined

At **{Payment Result Received}**, if the **Payment Gateway** reports that the payment was declined,

1. **System** shows the decline and the remaining hold time.
2. Resume the basic flow at **{Pay the Full Table Fee}**.

AF-6 Consent Refused

At **{Complete the Customer Profile}**, if the **Customer** declines the data-collection consent,

1. **System** explains that the booking cannot continue without it, releases the hold and sets the booking to Cancelled.
2. The use case ends.



AF-7 Invalid Profile Data

At {Complete the Customer Profile}, if the name is empty or the phone number is not a valid Thai mobile number,

1. **System** marks the field and asks for a correction.
2. Resume the basic flow at {Complete the Customer Profile}.

Exceptional Flows:

EF-1 Hold Expires Before Payment

At any point between {Hold the Table} and {Payment Result Received}, if the hold period of 15 minutes ends without a payment result (**Time** actor),

1. **System** releases the table, sets the booking to Expired and refreshes the zone map for every customer.
2. **System** informs the **Customer** by a LINE message that the hold has expired and that the table may be selected again if still available (subflow {Notify the Customer by LINE}).
3. The use case ends.

EF-2 Payment Result Arrives After the Hold Expired

At {Payment Result Received}, if a successful payment result arrives after the hold has expired,

1. **System** checks whether the table is still available.
 - a. If the table is still available, **System** places a new hold and resumes the basic flow at {Confirm the Booking}.
 - b. If the table has been taken, **System** requests a refund of the payment through the gateway's refund API, records the refund, and informs the **Customer** by LINE message that the table was taken in the meantime and the payment is being returned (subflow {Notify the Customer by LINE}). The use case ends.

EF-3 Payment Gateway Unreachable (Degraded Mode)

At {Pay the Full Table Fee}, if the **Payment Gateway** cannot be reached,

1. **System** keeps the hold, shows the shop's bank account and the full table fee, and asks the **Customer** to transfer the fee and attach the transfer slip.
2. **Customer** transfers the fee in a banking application and attaches the slip in the frontend.
3. **System** stores the slip with the booking, marks the booking "awaiting slip verification", extends the hold until the **Manager's** decision (degraded mode) and notifies the Manager.
4. **Manager** reviews the slip in the back-office.
 - a. If the **Manager** confirms the payment, **System** records the payment, sets the booking to Confirmed, marks the table as booked and resumes the basic flow at {Issue the E-Ticket}.



b. If the **Manager** rejects the slip, or no slip is attached before the hold ends, **System** releases the table, sets the booking to Expired and informs the **Customer** by LINE message (subflow {**Notify the Customer by LINE**}). The use case ends.

EF-4 Payment Result Not Received

At {**Payment Completed**}, if no payment result has arrived within 60 seconds after the gateway's checkout reported completion,

1. **System** queries the gateway's payment status API every 10 seconds until a result is known or the hold ends.
 - a. If the payment is confirmed, resume the basic flow at {**Confirm the Booking**}.
 - b. If the hold ends first, EF-1 applies; any payment result that arrives later follows EF-2.

EF-5 LINE Login Fails or Is Cancelled

At {**LINE Login Result**}, if LINE Login fails or the **Customer** cancels it,

1. **System** explains that reservation requires a LINE account and returns to the Rich Menu.
2. The use case ends.

EF-6 Confirmation Message Cannot Be Sent

At {**Confirmation Delivery Result**}, if the LINE Messaging API rejects the message or the account's push quota is exhausted,

1. **System** records the failure and retries three times over five minutes.
2. The use case ends normally; the booking is Confirmed.



Use Case Name: Check In Using Digital QR Ticket	ID: UC-02	Importance Level: High
Primary Actor: Front Staff	Use Case Type: Business / Transactional	
Stakeholders and Interests: Customer: Wants quick check-in and access to the correct reserved table. Staff: Wants to verify reservations quickly and prevent duplicate check-ins. Manager: Wants accurate attendance information and a live view of table occupancy and of no-show tables released for resale. Time: Triggers the automatic no-show marking at the end of the grace period.		
Brief Description: Customer arrives at the door and shows the e-ticket QR code on the phone. Front Staff scans the code with the back-office on a smartphone; System verifies the signed booking reference against the central record (Confirmed, tonight's round, not yet checked in, check-in window open) and displays the table, package, party size and payment status within 2 seconds. Front Staff checks the party size, confirms the entry, System sets the booking to Checked-in with time and staff account and updates the live view for the Manager. Invalid or duplicate tickets are escalated to the Manager; bookings not checked in by the end of the grace period are marked no-show and their tables released.		
Trigger: Customer arrives at the door and shows the e-ticket QR code on the phone. Type: External		
Relationships: Uses the Confirmed booking and e-ticket issued in UC-01 and the venue-issued back-office account of the Front Staff. Updates the live view seen by the Manager. Escalations are ruled on by the Manager; no-show bookings release their tables for walk-in resale or the waitlist.		
Precondition: - Customer holds a Confirmed booking for tonight's concert round and has the e-ticket on the phone (UC-01). - Front Staff is signed in to the back-office on a smartphone with a camera using a venue-issued username and password account. - The check-in window of the round is open: from 2 hours before the start until the end of the grace period.		



Postcondition:

Success (basic flow; AF-1 to AF-4; EF-3 accepted):

- Booking state = Checked-in with time and staff account; the party is seated at its table; the shared live view shows the table as occupied; any extra-person fee collected at the door is recorded.

Failure (AF-3 window not open; EF-1; EF-2; EF-3 refused; EF-5):

- Booking keeps its previous state; an escalation or a refusal is recorded.

No-show (AF-5; EF-4):

- Booking state = No-show; the table is released for walk-in resale or the waitlist; the full table fee is forfeited.

Normal Flow of Events:

{Present the E-Ticket}

1. The use case begins when the **Customer** arrives at the door and shows the e-ticket QR code on the phone.

{Scan and Verify}

2. **Front Staff** scans the QR code with the back-office on the phone. **{Booking Reference Obtained}**
3. **System** decodes the signed booking reference and verifies, against the central record, that the booking exists, is Confirmed, belongs to tonight's concert round, has not been checked in before, and that the check-in window is open (subflow **{Verify the Booking Reference}**). **{Verification Result}**
4. **System** displays the verification result within 2 seconds: table number and zone, table type and package, party size paid for, the name on the customer profile, and payment status "paid in full".

{Confirm Entry}

5. **Front Staff** confirms the number of guests present against the party size paid for. **{Party Size Confirmed}**
6. **Front Staff** confirms the entry; **System** sets the booking to Checked-in, records the time and the staff account, and shows the table number to guide the party.
7. **System** updates the live view of the back-office so that the **Manager** sees the table as occupied.

{Use Case Ends}

8. The use case ends when the party has been shown to the table.

Sub flows:



{Verify the Booking Reference}

1. **System** checks the signature of the booking reference.
2. **System** loads the booking and checks its status, round, table and prior check-in.
3. **System** compares the current time with the round's check-in window and grace period.
4. The subflow returns to the step that invoked it.

Alternate/Exceptional Flow:

Alternative Flows:

AF-1 Customer Cannot Show the QR Code

At **{Present the E-Ticket}**, if the **Customer** cannot show the e-ticket (no battery, no signal, deleted message),

1. **Front Staff** looks the booking up in the back-office by name, phone number or booking reference.
2. **System** shows the matching Confirmed bookings for tonight's round.
3. **Front Staff** selects the one whose name matches the customer's identification.
4. Resume the basic flow at **{Booking Reference Obtained}** with the selected booking.

AF-2 QR Code Unreadable

At **{Scan and Verify}**, if the camera cannot read the code (damaged screen, glare),

1. **Front Staff** types the booking reference printed under the QR code.
2. Resume the basic flow at **{Booking Reference Obtained}**.

AF-3 Check-In Window Not Open Yet

At **{Verification Result}**, if the current time is more than 2 hours before the start,

1. **System** shows the time at which the check-in window opens and does not change the booking.
2. **Front Staff** asks the **Customer** to return at that time. The use case ends.

AF-4 More Guests Than the Party Size Paid For

At **{Confirm Entry}**, if more guests are present than the party size paid for,

1. If the guests fit within the capacity of the table type, **System** records the new party size and no fee is due.
2. If they exceed the capacity, **System** computes and displays the extra-person fee for the additional persons.
3. **Front Staff** collects the fee at the door and records it as paid at the venue.
4. Resume the basic flow at **{Party Size Confirmed}**.

AF-5 Arrival After the Grace Period

At **{Verification Result}**, if the current time is more than 30 minutes after the concert start and the booking has not been checked in,



1. **System** shows that the booking is a no-show: the table has been released for walk-in guests or the waitlist and the full table fee is not refunded.
2. If the table is still free, **Front Staff** seats the party as a walk-in.
3. **System** records the late arrival against the booking. The use case ends.
4. Otherwise **Front Staff** informs the **Customer**. The use case ends.

Exceptional Flows:

EF-1 Ticket Already Used

At {**Verification Result**}, if the booking is already Checked-in,

1. **System** shows "already checked in" with the time and the staff account of the earlier check-in.
2. If the party is joining a group already seated at the same table (latecomers of the same booking), **Front Staff** records the additional guests against the booking and lets them through without a second check-in; if the guests exceed the capacity of the table type, AF-4 applies. The use case ends.
3. Otherwise **Front Staff** escalates the case to the **Manager** as a duplicate ticket (EF-3).

EF-2 Ticket for Another Round or an Unknown Booking

At {**Verification Result**}, if the reference is not found, its signature is invalid, or the booking belongs to a different concert round or was Transferred or Cancelled,

1. **System** shows the reason (unknown ticket, wrong date, transferred, cancelled).
2. **Front Staff** escalates the case to the **Manager** (EF-3).

EF-3 Escalation to the Manager

At EF-1 or EF-2, when **Front Staff** escalates an invalid ticket or a duplicate ticket,

1. **System** records the escalation with the scanned data, the reason and the staff account, and notifies the **Manager** on the shared view.
2. **Manager** examines the booking and payment records and rules: accept, with a note, or refuse.
3. If accepted, **System** sets the booking to Checked-in on the **Manager**'s decision and resumes the basic flow at {**Party Size Confirmed**}.
4. If refused, **System** records the refusal.
5. **Front Staff** informs the **Customer** that entry cannot be granted. The use case ends.

EF-4 No-Show Marking After the Grace Period

At the end of the round's grace period, for every Confirmed booking not yet Checked-in (**Time** actor),

1. **System** marks the booking as no-show automatically at the cutoff.
2. **System** releases the table for walk-in resale or the waitlist and records that the fee is forfeited.
3. The use case ends.

EF-5 Check-In Cannot Be Saved



-
- At {Confirm Entry}, if **System** cannot save the check-in when **Front Staff** confirms the entry,
1. **System** informs **Front Staff** that the check-in has not been confirmed; the booking stays Confirmed and the live view is unchanged.
 2. **Front Staff** retries.
 3. **System** checks the existing check-in status first so that no duplicate check-in is recorded.
 4. Resume the basic flow at {Party Size Confirmed}.



Use Case Name: Create Concert Event	ID: UC-03	Importance Level: High
Primary Actor: Manager	Use Case Type: Business / Creation	
Stakeholders and Interests: Manager: Wants to publish a concert round quickly with the right zone map, table types, package prices and booking-open time, priced according to the venue's policy, and to see it in the back-office live view so that bookings and check-ins can be handled. Customer: Wants accurate round information, table availability and prices before reserving (UC-01). Front Staff: Wants the round's check-in window and grace period defined so that check-in works on the night (UC-02).		
Brief Description: Manager creates a concert round in the back-office: enters the concert details, date, start time and booking-open time, chooses the venue zone map and marks the tables for sale, sets the package price and table fee of each table type and the extra-person fee, reviews the round as Customers will see it and publishes it. From its booking-open time the round is open for reservation in UC-01, and its check-in window and grace period drive check-in in UC-02.		
Trigger: Manager wants to open a new concert round for booking. Type: External		
Relationships: Requires the venue zone map (zones, tables, table types and seat-view photographs) created in UC-04 and the venue-issued back-office account of the Manager. Produces the concert round used in UC-01 (rounds, table map, fees) and in UC-02 (check-in window and grace period). The live view of published rounds is shared by the Manager.		
Precondition: - Manager is signed in to the back-office using a venue-issued username and password account. - An Active venue zone map with zones, tables and table types exists (UC-04). - The venue's pricing policy for package, table and extra-person fees is known to the Manager.		
Postcondition: Success (basic flow; AF-2 copy; AF-3 edit): - Round state = Published; the round appears to Customers in UC-01 from its booking-open time; its check-in window and grace period are available to UC-02; the live view shows the round.		



Draft (AF-1 save as draft; AF-4 unpublish; EF-1 not corrected):

- Round state = Draft; the round is not visible to Customers and no booking is possible.

Unchanged (EF-2 cannot be saved):

- No round is created or changed; previously published rounds are unaffected.

Normal Flow of Events:

{Open Round Creation}

1. The use case begins when the **Manager** opens the concert rounds section of the back-office and chooses to create a new round.
2. **System** displays a new, empty round form.

{Enter the Concert Details}

3. **Manager** enters the concert name, the artist or programme, the date, the doors-open time and the start time.
4. **Manager** enters the booking-open time from which **Customers** may reserve.
5. **System** derives the check-in window of the round (from 2 hours before the start until 30 minutes after the start) and displays it for confirmation. {Schedule Set}

{Choose the Zone Map}

6. **Manager** selects the venue zone map to use for the round.
7. **System** displays the zones, tables and table types of the map.
8. **Manager** marks any table that is not for sale in this round.

{Set the Prices}

9. **Manager** sets the package price and the full table fee of each table type and the extra-person fee.
10. **System** computes and displays the number of tables for sale and the capacity of each zone.

{Prices Set}

{Review and Publish}

11. **Manager** chooses to review the round.
12. **System** validates the round (subflow {Validate the Round}). {Validation Result}
13. **System** shows a preview of the round as **Customers** will see it in UC-01.
14. **Manager** publishes the round.
15. **System** saves the round as Published, makes it visible to **Customers** from its booking-open time and updates the back-office live view. {Round Saved}
16. The use case ends.

Sub flows:

{Validate the Round}



1. **System** checks that the doors-open time, the start time and the booking-open time are in order and that the booking-open time is before the start.
2. **System** checks that every table type for sale has a package price and a table fee.
3. **System** checks that no other Published round overlaps the same date and time.
4. The subflow returns to the step that invoked it.

Alternate/Exceptional Flow:

Alternative Flows:

AF-1 Save as Draft

At any point before **{Review and Publish}**, if the **Manager** chooses to save without publishing,

1. **System** saves the round as Draft; it is not visible to **Customers**.
2. The use case ends.

AF-2 Copy from an Earlier Round

At **{Open Round Creation}**, if the **Manager** chooses to copy an earlier round,

1. **System** pre-fills the form with the earlier round's zone map, table selection and prices; the date, the times and the booking-open time are left blank.
2. Resume the basic flow at **{Enter the Concert Details}**.

AF-3 Edit a Published Round

At **{Open Round Creation}**, if the **Manager** opens a Published round instead of creating a new one,

1. **System** shows the round with the number of Confirmed bookings on each table.
2. If the round has no Confirmed booking, **Manager** may change any field; resume the basic flow at **{Enter the Concert Details}**.
3. Otherwise **System** allows changes only to fields that do not affect existing bookings (concert details, tables not yet booked) and prevents changes to the date, the start time, the prices and the removal of tables with Confirmed bookings, displaying the affected bookings.
4. Resume the basic flow at **{Review and Publish}**.

AF-4 Unpublish a Round

At **{Review and Publish}**, if the **Manager** withdraws a Published round that has no Confirmed booking,

1. **System** sets the round back to Draft and removes it from UC-01.
2. The use case ends.

Exceptional Flows:

EF-1 Validation Fails

At **{Validation Result}**, if the round does not pass validation,

1. **System** marks the invalid fields and states the reason (times out of order, unpriced table type, overlapping round).



2. **Manager** corrects the fields.
3. Resume the basic flow at **{Review and Publish}**.

EF-2 Round Cannot Be Saved

At **{Round Saved}**, if **System** cannot save the round,

1. **System** informs the **Manager** that the round has not been published and keeps the entered data in the form.
2. **Manager** retries.
3. **System** checks whether the round already exists so that no duplicate round is created and saves the round.
4. The use case ends when the round is saved or the **Manager** abandons the form.

EF-3 Zone Map Changed While Editing

At **{Choose the Zone Map}**, if the zone map has been changed in the back-office since the form was opened,

1. **System** reloads the map and highlights the tables that changed.
2. Resume the basic flow at **{Choose the Zone Map}**.



Use Case Name: Create Venue Zone Map	ID: UC-04	Importance Level: Medium
Primary Actor: Manager	Use Case Type: Business / Creation	
Stakeholders and Interests: Manager: Wants the digital zone map to match the physical venue so that every concert round can be set up on it without rework, with the zones, tables, table types and seat-view photographs ready to select when creating a concert round (UC-03), and the layout reflecting the venue's capacity and policy. Customer: Wants accurate table positions, table types and seat-view photographs before reserving (UC-01). Front Staff: Wants the live floor plan used at check-in to match the actual room layout (UC-02).		
Brief Description: Manager creates or edits the venue zone map in the back-office: defines the zones, places the tables on the map, gives each table a table number, a table type and a seating capacity, uploads the seat-view photograph of each table, and activates the map. System validates the map (named zones, unique table numbers, every table typed, sized and photographed) before it can be activated. An Active zone map can be selected by the Manager when creating a concert round in UC-03 and is rendered as the floor plan seen by Customers in UC-01 and by Front Staff in UC-02.		
Trigger: Manager wants to set up the venue layout for the first time or change the layout of the venue. Type: External		
Relationships: Requires the venue-issued back-office account of the Manager. Produces the venue zone map (zones, tables, table types and seat-view photographs) that UC-03 requires when a concert round is created and that is rendered in UC-01 and UC-02.		
Precondition: - Manager is signed in to the back-office using a venue-issued username and password account. - The venue's table types (e.g. standard, high table, VIP sofa) and their seating capacities are known to the Manager.		
Postcondition: Success (basic flow; AF-1 edit; AF-2 copy):		



- Zone map state = Active; the map can be selected in UC-03 and is rendered in UC-01 and UC-02 for the rounds that use it.

Draft (AF-3 save as draft; EF-1 not corrected):

- Zone map state = Draft; the map cannot be selected in UC-03.

Unchanged (AF-1 blocked change; EF-2 cannot be saved):

- No zone map is created or changed; rounds already using the map are unaffected.

Normal Flow of Events:

{Open Map Editor}

1. The use case begins when the **Manager** opens the venue section of the back-office and chooses to create a new zone map.
2. **System** displays a new, empty map editor.

{Define the Zones}

3. **Manager** draws the zones of the venue (e.g. front stage, middle, bar) and names each zone.

{Place the Tables}

4. **Manager** places each table in its zone on the map.
5. **Manager** enters, for each table, its table number, table type and seating capacity.
6. **Manager** uploads the seat-view photograph of each table. **{Tables Defined}**
7. **System** displays the number of tables and the total capacity of each zone.

{Validate and Activate}

8. **Manager** chooses to activate the zone map.
9. **System** validates the zone map (subflow **{Validate the Zone Map}**). **{Validation Result}**
10. **System** shows a preview of the map as **Customers** will see it in UC-01.
11. **Manager** confirms the activation.
12. **System** saves the zone map as Active and makes it available for selection in UC-03. **{Map Saved}**
13. The use case ends.

Sub flows:

{Validate the Zone Map}

1. **System** checks that every zone has a name and contains at least one table.
2. **System** checks that table numbers are unique across the whole map.
3. **System** checks that every table has a table type, a seating capacity and a seat-view photograph.
4. The subflow returns to the step that invoked it.



Alternate/Exceptional Flow:

Alternative Flows:

AF-1 Edit an Active Zone Map

At {Open Map Editor}, if the **Manager** opens an Active zone map instead of creating a new one,

1. **System** shows the map with the Published concert rounds that use it and the number of Confirmed bookings on each table.
2. If no Published round uses the map, **Manager** may change any zone or table; resume the basic flow at {Define the Zones}.
3. Otherwise **System** allows changes only to fields that do not affect existing bookings (zone names, seat-view photographs, tables with no Confirmed booking) and prevents the removal or relocation of tables with Confirmed bookings, displaying the affected bookings.
4. Resume the basic flow at {Validate and Activate}.

AF-2 Copy an Existing Zone Map

At {Open Map Editor}, if the **Manager** chooses to copy an existing zone map,

1. **System** pre-fills the editor with the zones, tables, table types and seat-view photographs of the existing map as a new Draft.
2. Resume the basic flow at {Define the Zones}.

AF-3 Save as Draft

At any point before {Validate and Activate}, if the **Manager** chooses to save without activating,

1. **System** saves the zone map as Draft; it cannot be selected in UC-03.
2. The use case ends.

Exceptional Flows:

EF-1 Validation Fails

At {Validation Result}, if the zone map does not pass validation,

1. **System** marks the invalid zones or tables and states the reason (unnamed or empty zone, duplicate table number, missing table type, capacity or photograph).
2. **Manager** corrects the zones or tables.
3. Resume the basic flow at {Validate and Activate}.

EF-2 Zone Map Cannot Be Saved

At {Map Saved}, if **System** cannot save the zone map,

1. **System** informs the **Manager** that the map has not been saved and keeps the entered data in the editor.
2. **Manager** retries.
3. **System** checks whether the map already exists so that no duplicate map is created and saves the map.



4. The use case ends when the map is saved or the **Manager** abandons the editor.

EF-3 Seat-View Photograph Cannot Be Uploaded

At {Place the Tables}, if a seat-view photograph cannot be uploaded,

1. **System** informs the **Manager** that the upload failed and keeps the other details of the table.
2. **Manager** retries the upload or chooses another photograph.
3. Resume the basic flow at {Place the Tables}.



Functional Requirements

User Authentication & Login System

- The system shall allow customers to authenticate using LINE Login.
- The system shall allow customers to access only their own reservations and digital tickets.
- The system shall authenticate all internal role accounts (Front Staff and Manager) with a username and password issued by the venue, separately from customer LINE Login.
- The system shall allow all users to log out securely.

Venue Zone Map & Concert Round Management System

- The system shall allow admins to create and edit venue zone maps with zones, tables, table types, seating capacity and seat-view photographs, and to activate a map for use in concert rounds.
- The system shall validate a zone map before activation: every zone named and non-empty, table numbers unique, and every table with a table type, seating capacity and seat-view photograph.
- The system shall prevent the removal or relocation of a table that has confirmed bookings in a published concert round.
- The system shall allow managers to create a concert round with concert details, date, times and booking-open time, select an active zone map, mark tables not for sale, set package prices, table fees and the extra-person fee, and publish the round.
- The system shall validate a concert round before publishing and prevent overlapping published rounds.
- The system shall display real-time table availability, seat-view photographs, table type, seating capacity and zone for each table of a published concert round.

Table Reservation System

- The system shall allow customers to select and reserve a specific available table.
- The system shall validate party size and prevent conflicting bookings.
- The system shall display reservation details, cutoff time, and booking conditions before confirmation.
- The system shall save confirmed reservations and update table availability.

Digital Ticket & Check-In System

- The system shall generate a digital QR ticket for each confirmed reservation.



- The system shall allow customers to view their tickets in the web app.
- The system shall allow staff to scan tickets and confirm valid reservations.
- The system shall prevent invalid or duplicate check-ins and update checked-in tables to occupied.
- The system shall allow staff to update table statuses on the real-time floor plan.
- The system shall allow staff to assign an available table to a walk-in customer and record it as a walk-in booking.
- The system shall prevent a walk-in assignment to a table that is already held by a confirmed reservation.

Arrival & Notification System

- The system shall send cutoff reminders through LINE.
- The system shall allow customers to select “I’m on my way” or request a 30-minute extension.
- The system shall allow staff to approve or reject extension requests and notify customers of the result.
- The system shall change the cutoff only after an extension is approved.
- The system shall expire reservations and release tables after the effective cutoff if customers have not checked in.



Non-functional Requirements

Operational

- The system shall support desktop and mobile devices.
- The customer web app shall operate within LINE Messenger on Android and iOS.
- The system shall back up reservation and venue data daily.

Performance

- The system shall complete at least 95% of booking and QR validation requests within 2 seconds under a test load of 100 concurrent users, excluding external service and network delays.
- The system shall display table status updates within 3 seconds under normal network conditions.
- The system shall target monthly availability of at least 99.5%.

Security

- The system shall verify customer identity through LINE Login.
- The system shall enforce role-based access for Manager, Staff, and Customer.
- The system shall use HTTPS and protect customer information and authentication credentials.
- The system shall prevent unauthorized access to reservations and digital tickets.

Cultural and Legal

- The system shall provide a Thai-language interface.
- The system shall display booking and cutoff times using Thailand's local time zone.
- The system shall handle personal information in accordance with Thailand's PDPA and provide a privacy notice.

Usability

- The system shall provide a responsive interface with clear navigation.
- The system shall clearly display table availability, cutoff times, and extension request statuses.
- The system shall provide understandable error messages.
- The system shall allow customers to retrieve their reservations and tickets easily.



Architecture Decision Records (ADRs)

ADR-01: Frontend Architecture & Client Channel

Context

Target users primarily interact via mobile phones while coordinating nightlife plans on LINE. Requiring an App Store/Play Store download would cause high user drop-off. The client must support mobile responsiveness and rich interactive graphics for table selection.

Decision

Use Next.js (React) bundled as a LINE Front-end Framework (LIFF) Web App, with fallback access via standard mobile browsers.

Status

Accepted

Consequences

- Eliminates app install friction; customers book directly inside the LINE app.
- Allows seamless integration with LINE Login and LINE Messaging API.
- Requires compliance with LIFF webview constraints and in-app browser caching behaviors.



ADR-02: Real-Time Floor Plan Communication

Context

When multiple customers look at the floor plan simultaneously on busy nights, table statuses must update instantaneously to prevent double-booking conflicts and outdated views.

Decision

Implement WebSocket (or Server-Sent Events - SSE) for real-time table status broadcasting, combined with short-term Redis distributed locks for table selection checkout.

Status

Accepted

Consequences:

- Delivers sub-second UI updates when tables are locked, booked, or released.
- Avoids wasteful client polling against the backend server.
- Requires maintaining persistent connection pools and managing socket disconnect/reconnect states on mobile networks.



ADR-03: Backend Language & Framework

Context

The system requires rapid development cycles while supporting RESTful APIs, real-time bi-directional communication for floor plan updates, seamless integration with the LINE Messaging API/LIFF SDK, and scheduled background tasks to automatically release expired table reservations past the cutoff window. Options considered include Go, Java, and Node.js (JavaScript). Given that the frontend is built using Next.js/React, adopting a unified language stack across client and server significantly streamlines development.

Decision

Use Node.js (JavaScript) with the Express.js framework for the core API services, combined with Socket.io for real-time WebSocket communication and node-cron for background cutoff scheduling.

Status

Accepted

Consequences

- Enables a unified full-stack JavaScript environment, allowing the team to share utility logic, minimize context switching, and accelerate feature delivery.
- Leverages Node.js's event-driven, non-blocking I/O model to handle concurrent client connections efficiently during peak reservation hours.
- Because JavaScript is dynamically typed, the backend requires robust runtime request validation and clear documentation to prevent data type mismatches.



ADR-04: Notification Engine

Context

Reminders before cutoff times are critical to reducing lost tables. Traditional SMS is costly per transaction, whereas email is rarely checked by nightlife patrons in real time.

Decision

Use LINE Messaging API (Push Messages / Flex Messages) as the primary notification channel, supplemented by Web Push notifications.

Status

Accepted

Consequences

1. Near 100% open rate with interactive rich cards ("On My Way" / "Postpone 30 mins").
2. Low messaging cost within standard official account quotas.
3. Requires users to add or link the venue's LINE Official Account (OA).



ADR-05: Interactive Floor Plan Rendering

Context

The floor plan needs to support responsive rendering, pinch-to-zoom, zone highlighting, and seat view popups without lagging on mid-range smartphones.

Decision

Use HTML5 Canvas / SVG (via Fabric.js or Konva.js) embedded within the Next.js client.

Status

Accepted

Consequences

- Smooth vector scaling and interactive zone/table hit-testing.
- Managers can visually manipulate coordinates for tables and stages.
- Requires custom responsive coordinate mapping to ensure layouts adapt across various mobile screen sizes.



ADR-06: Primary Database Engine

Context

The system needs to store diverse data structures, including flexible venue floor plan layouts (coordinates, shapes, multi-table zone configs, table perspective photos), booking transaction records, customer profiles, and time-stamped status transitions. Traditional relational databases (RDBMS) enforce rigid table schemas that make storing deeply nested floor layout geometries and dynamic table attributes cumbersome. Additionally, the backend runtime is Node.js/JavaScript, making native JSON/BSON document handling particularly advantageous for rapid development.

Decision

Use MongoDB as the primary NoSQL document database, accessed via the Mongoose Object Data Modeling (ODM) library in the Node.js backend.

Status

Accepted

Consequences

- Flexible Schema for Layouts: Enables storing complex, nested JSON objects (such as floor plan coordinate maps, zone definitions, and table media links) directly without complex multi-table joins.
- Seamless Node.js Integration: Documents map directly to JavaScript objects (JSON) reducing serialization overhead and development friction.
- Native TTL (Time-To-Live) Indexes: Provides built-in TTL indexing capabilities, which can automatically handle temporary reservation locks or draft bookings that expire if not checked out within a few minutes.
- Horizontal Scalability: Supports horizontal partitioning (sharding) should the platform expand to serve multiple nightlife venues concurrently.



ADR-07: Internal Role Authentication (Front Staff, Manager)

Context

Customers reach the web app through LINE and are authenticated with LINE Login, as decided in ADR-01. Front Staff and Managers use internal accounts created by the venue, and front-of-house staff work from shared devices. Binding those accounts to personal LINE profiles would tie venue access to individual social accounts and make shift handover and staff turnover difficult. Options considered: LINE Login for every role, venue-issued username and password accounts, and an external identity provider.

Decision

Use venue-issued username and password accounts with JWT sessions for all internal roles (Front Staff and Manager), kept separate from the customer LINE Login flow. Each account carries a role that determines which back-office functions it may use (e.g. check-in for Front Staff; zone map and round creation for the Manager). The Manager creates, updates, and deactivates internal accounts from the venue configuration.

Status

Accepted

Consequences

- Venue access stays under the owner's control and survives staff turnover, because accounts are not tied to personal LINE profiles.
- Requires the system to implement password storage, session expiry, and account lockout instead of delegating all authentication to LINE.
- Two authentication paths must be maintained and tested, one for customers and one for internal roles.